Hands-On Machine Learning

Chapter 3 — Classification

Precision · Recall · ROC · Multiclass · Multilabel

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

The book's MNIST is not downloadable in this runtime environment, so the green **Output (real run)** boxes came from an actual run on the real `digits` data from sklearn (1797 images of 8×8): the same idea but an easier problem, so the numbers are better than MNIST's. The MNIST numbers from the book (3rd edition) are written in orange boxes and marked "from the book".

---

1 The MNIST dataset

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame=False)   # as_frame=False: return NumPy arrays, not a
DataFrame
X, y = mnist.data, mnist.target
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

From your notebook

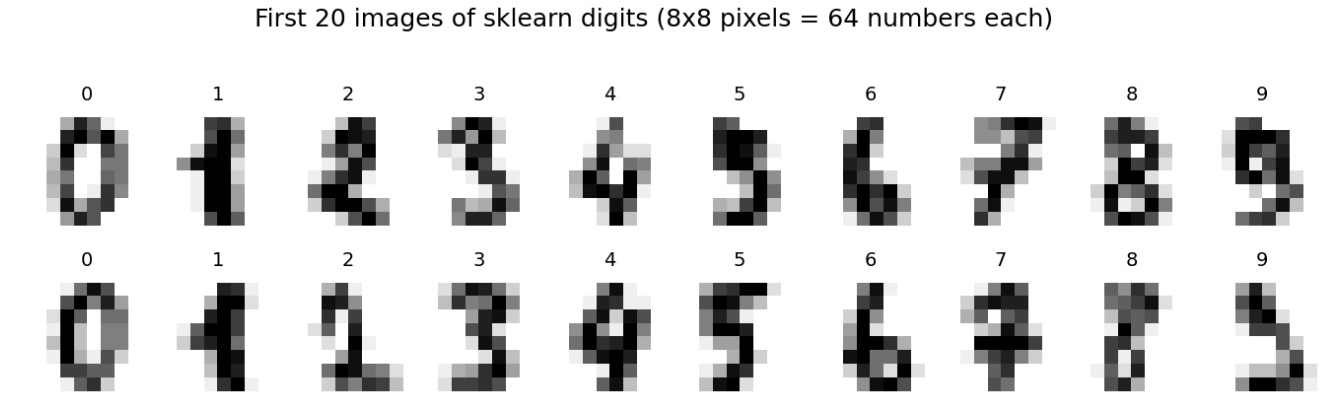
The first thing in this chapter, we will use MNIST (a dataset), which is a set of 70,000 small images of handwritten digits (written by many people). Each image represents a digit (from 0 to 9), and each image is 28 × 28 = 784 pixels, and each pixel has a numeric value from 0 to 255 (the pixel color intensity: from 0 white to 255 black).

Remark: in `sklearn.datasets` there are 3 kinds of functions: `fetch_*` (download large real data), `load_*` (small ready-made data), `make_*` (artificial data for testing).

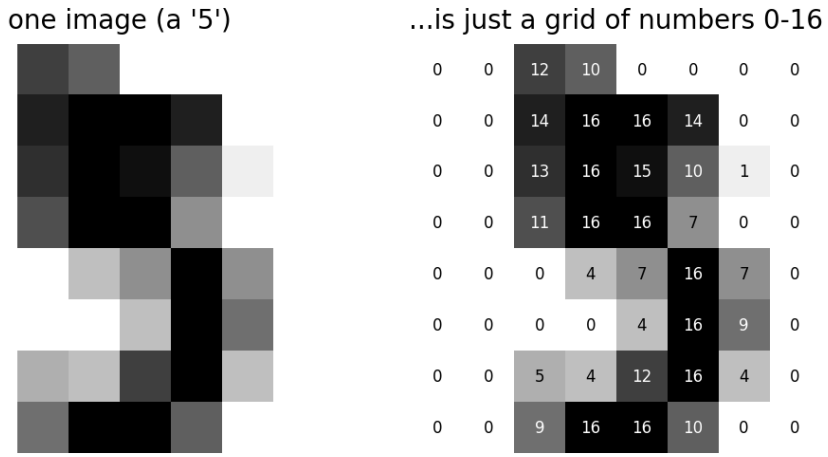
From your notebook

`as_frame=False` because image data is handled better as NumPy arrays (a DataFrame is not suitable for images). MNIST is **already split and shuffled**: the first 60,000 = training, and the last 10,000 = test. Shuffling is important because some algorithms are sensitive to the order of the data, and work badly if they get many similar examples in a row.

`X_train` = the training images, `X_test` = the test images, `y_train` = the training answers (labels), `y_test` = the test answers.



The examples here are from the real `digits` (8×8) instead of MNIST for the sake of illustration; MNIST images are 28×28 and the idea is the same.



The image is just a grid of numbers: each number represents a pixel's intensity (here 0-16; in MNIST from 0-255).

Addition

- The `y` in MNIST returns **strings** (`'5'` not `5`), so in your code you write `y_train == '5'`. And if you want numbers: `y.astype(np.uint8)`.
- Each row of `X` is one image flattened into a vector of length 784. And if you want to display an image: `some_digit.reshape(28, 28)` and then `plt.imshow(..., cmap="binary")`.
- MNIST is called the "hello world" of ML, because everyone who learns classification tries a new algorithm on it.
- In the real run here: `digits` has **1797** images, we split them into 1437 training and 360 test (stratified).

2 Training a Binary Classifier (5 or not 5)

From your notebook

In the previous chapter the model predicted a number (the house price, Regression). Here in Classification the model predicts a **class** (category / class). That is, we want the model to predict: is this digit 5 or not 5? And so on. And it is a Binary classification because it answers with `True` or `False`.

Steps:

1. **Simplify the problem:** instead of the model knowing all 10 digits (0 to 9), we make it answer one question: is the image the digit 5 or not? That is, we try to detect only one digit (we consider 5 as True and everything else False).

```
y_train_5 = (y_train == '5')    # True for 5, False for any other digit
y_test_5 = (y_test == '5')

from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)

some_digit = X_train[y_train_5][0]    # the first real '5' in the training set
print(sgd_clf.predict([some_digit]))
```

From your notebook

2. Create the new Target. Here we convert the original labels (0, 1, 2, 3, ... 9) to `True` (if the image is 5) and `False` (if any other digit). So the model does not need to know the digit exactly, it only needs to know whether it is 5 or not 5 (because we are now building a 5-detector).

3. Choose the classifier: `SGDClassifier` (a classification algorithm). SGD = Stochastic Gradient Descent. The basic idea in it is that during training it deals with training instances **one by one** instead of processing all the data at once, which makes it suitable for large datasets and suitable for online learning.

From your notebook

4. Train the classifier: here we tell the model: `X_train` (the images) and `y_train_5` (is each image a 5 or not). So it learns the relationship between the shape of the pixels and True (5) and False (not 5). That is, after training it can see a new image and decide: is this a 5?

5. Use the model for prediction: `sgd_clf.predict([some_digit])` ← if it gives True = 5, and if False = not 5.

Output (real run)

```
[ True]      # decision score = 178.9 > 0
```

Addition

In the book the model correctly predicted that `some_digit` is 5 (`array([True])`). And a remark: `SGDClassifier` uses `loss="hinge"` by default, so it behaves like a Linear SVM, and it can become logistic regression if you set `loss="log_loss"`.

3 Evaluating the Classifier: Accuracy

From your notebook

Evaluating a classifier is often much trickier than evaluating a regressor, so there are many measures to evaluate performance.

One good way to evaluate a model is to use Cross-Validation. You may find the accuracy high, but that does not always mean the model is good. An example from the book: a **dummy classifier** classifies every picture in the most frequent class (non-5): it will get an accuracy of about **90%** because only about 10% of the images are 5, so it will be right about 90% of the time.

This shows that accuracy is generally not the preferred performance measure for classifiers, especially when dealing with **skewed** (imbalanced) datasets, i.e. when some classes are much more frequent than others.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

```
from sklearn.model_selection import cross_val_score
from sklearn.dummy import DummyClassifier

print(cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy"))
dummy_clf = DummyClassifier()                 # always predicts the most frequent class
print(cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy"))
```

Output (real run)

```
[0.9833 0.9896 0.9833]   # SGD
[0.8977 0.8998 0.8998]   # dummy: learns NOTHING and still gets ~90%
```

Addition

From the book (MNIST): SGD gives `[0.95035, 0.96035, 0.9604]` (more than 95% in every fold!), and the dummy gives `[0.90965, 0.90965, 0.90965]`. So the 95% of the SGD is not very impressive: the difference from a model that learns nothing is only 5%.

And in the real run on `digits`: SGD ≈ 98.5% and the dummy ≈ 89.9%. The same lesson: you must compare against the dummy baseline.

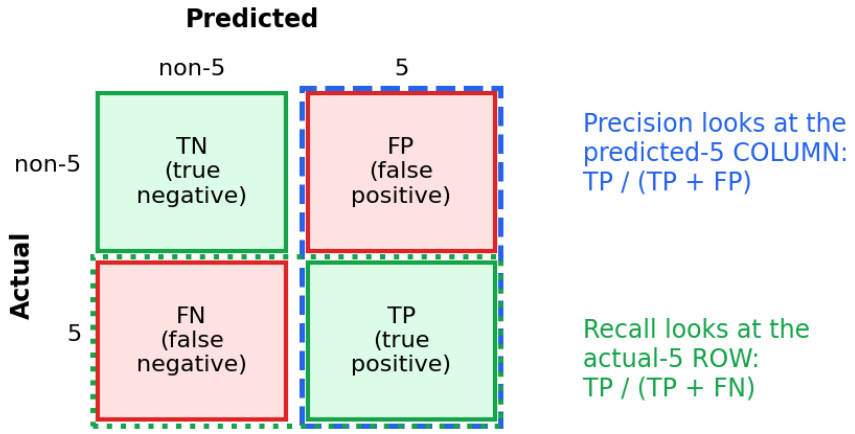
The book also shows you how to do cross-validation yourself with `StratifiedKFold` (3 folds): each fold trains a `clone` of the classifier and measures the percentage of correct predictions, and gives the same numbers; useful if you need more control over the CV process.

4 Confusion Matrix

From your notebook

A much better way to evaluate the performance of a classifier (Confusion Matrix). Its idea: we consider that we have a Binary classifier answering 5 or not 5, so each image has two things: **Actual** (is it really 5 or not) and **Predicted** (the model said 5 or not). When we compare the two we get only **4 cases**:

- `T` = True, `F` = False, `P` = Positive, `N` = Negative.
- The CM counts how many times the model confused class A with class B for every possible combination. A book example: if you want to know how many times the model confused an image of "5" and considered it not 5 (and vice versa).



The rows = reality, and the columns = the model's prediction. Precision looks at the predicted-5 column, and Recall looks at the actual-5 row.

From your notebook

To compute the CM we must make predictions first. We can use `cross_val_predict`:

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)

from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_train_5, y_train_pred)
print(cm)
```

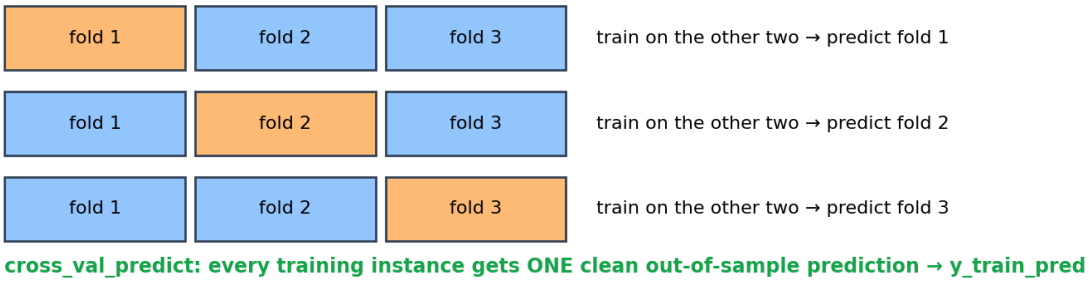
Output (real run)

```
[[1288    4]
 [  17  128]]
```

From your notebook

Note: `cross_val_predict` works like `cross_val_score`, but instead of returning a score it returns **predictions** for every instance in `X_train`. It splits the data into 3 folds: each time it trains on two parts and makes predictions on the third part, which the model has not seen. And in the end every training instance has an out-of-sample prediction (from a model that was not trained on this instance); and this is very important so that the predictions are **fair** when we use them in evaluating the model.

After that we now have `y_train_5` which is the actual labels, and `y_train_pred` which is the predicted labels. These numbers each indicate something, and a lot of information comes out of them. Because the CM provides a lot of information, but sometimes you prefer a more concise measure (Precision and Recall).



The idea of `cross_val_predict`: every prediction comes from a model that has not seen that instance.

Addition

Reading the real numbers (digits): **TN = 1288** (non-5 and said non-5 ✓), **FP = 4** (non-5 and said 5 ✗ "false alarm"), **FN = 17** (5 and said non-5 ✗ "missed the 5"), **TP = 128** (5 and said 5 ✓). The total = 1437 = the whole training set.

From the book (MNIST, 60,000 images):

```
array([[53892,   687],
       [ 1891,  3530]])
```

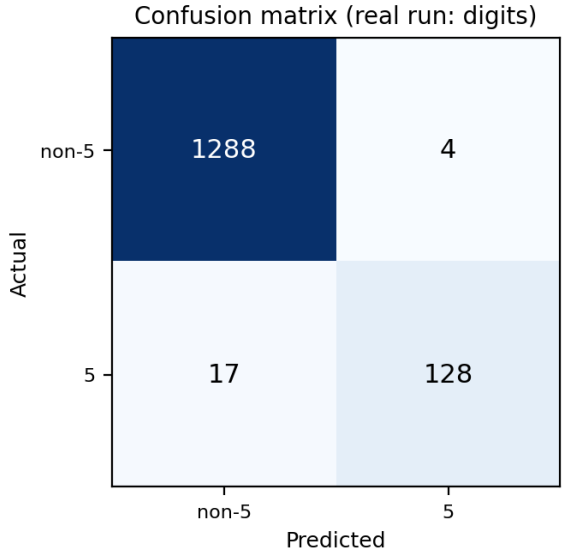
Addition

TN = 53,892 · FP = 687 · FN = 1,891 · TP = 3,530.

If the model were perfect (`confusion_matrix(y_train_5, y_train_5)`) it would have only TP and TN, and all the FP and FN would be zeros:

Output (real run)

```
[[1292    0]
 [   0  145]]
```



The real confusion matrix on `digits`.

5 Precision and Recall

```
from sklearn.metrics import precision_score, recall_score
print(precision_score(y_train_5, y_train_pred))
print(recall_score(y_train_5, y_train_pred))
```

From your notebook

Precision (the accuracy of the model's positive predictions) is the accuracy of the classifier's positive predictions. That is, it answers the question: of all the images the model said are 5, how many were really 5? (the correct ones among the predictions)

$$\text{precision} = \frac{TP}{TP + FP}$$

From your notebook

(All the ones the model said are Positive = all the predictions.)

But this alone is not enough: the problem is that you can easily achieve `precision = 100%` without the model being good. **Example:** I have 1000 images, of which 100 are really 5 and 900 are not 5, and I made a very dumb model that says non-5 for all images except one image it is very sure is 5, and the image really was 5 (`TP=1, FP=0` ← `precision = 1/(1+0) = 100%`). Is the model good? Of course not! Here **Recall** comes in.

Recall asks a different question: of all the images that are really 5, how many did the model detect? (also called sensitivity or true positive rate `TPR`)

$$\text{recall} = \frac{TP}{TP + FN}$$

From your notebook

(All the real Positives.)

Output (real run)

```
0.9697
0.8828
```

From your notebook

- The first means: of all the images the model said are 5, about **97.0%** really were 5.
- The second means: of all the images that are really 5, the model detected about **88.3%** of them.

Addition

The calculation from the CM numbers: precision = 128 / (128 + 4) = **0.9697**, and recall = 128 / (128 + 17) = **0.8828** (the same output).

From the book (MNIST): precision = 3530 / (3530 + 687) = **0.8371** and recall = 3530 / (3530 + 1891) = **0.6512**. Notice that the model does not look as impressive as the accuracy (96%) suggested: when it says it is a 5 it is right only 83.7% of the time, and it misses 35% of the 5s. (On `digits` the problem is easier so the numbers are higher.)

6 F1 Score

From your notebook

Sometimes you need one number that combines Recall and Precision: the F1 score, which is the **Harmonic Mean** of Precision and Recall.

$$F_1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

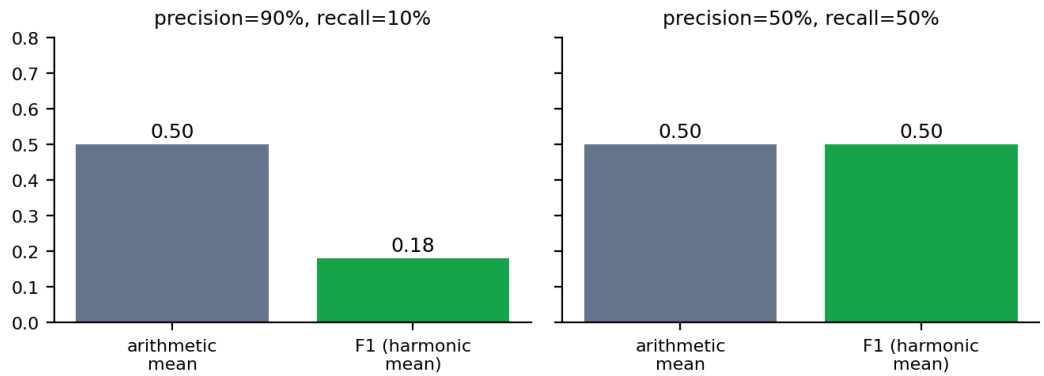
```
from sklearn.metrics import f1_score
f1_score(y_train_5, y_train_pred)
```

Output (real run)

```
0.9242
```

From your notebook

Why don't we use the ordinary mean? Because F1 penalizes if one of Recall or Precision is very low. For example if `Precision = 90%` and `Recall = 10%` the ordinary mean would say 50% (as if the model were good) but the F1 would be low (about 18%), because the model must be good in both together to get a high F1.



The ordinary mean deceives (0.50), but F1 penalizes the imbalance between precision and recall (0.18).

From your notebook

Is F1 always the best metric? No. Sometimes you care about Precision and sometimes about Recall. **Example:** you want to make sure that the video the model allows is safe for children; you can accept a model that rejects a lot of safe videos (low Recall) as long as it only allows the really safe videos (high Precision). Here Precision is the most important. And so on.

Addition

- From the book: F1 for the SGD = **0.7325**. And it prefers models whose precision and recall are close. (On `digits`: F1 = 0.924.)
- The opposite example from the book: if you are building a shoplifting detection system from surveillance cameras, a high recall is better (you accept many false alarms as long as you do not miss a real theft) even if the precision is only 30%.
- The rule: **you cannot have both together**, and this is what is called the Precision/Recall Trade-off (the next section).

7 Precision/Recall Trade-off

From your notebook

Increasing Precision usually leads to a decrease in Recall, and vice versa. Why? Because the classifier has something called a **Threshold**: the model does not decide directly 5 or not 5, it first computes a Score for each image. For example:

- Image A ← score = 5000
- Image B ← score = 3000
- Image C ← score = 1000
- Image D ← score = −500

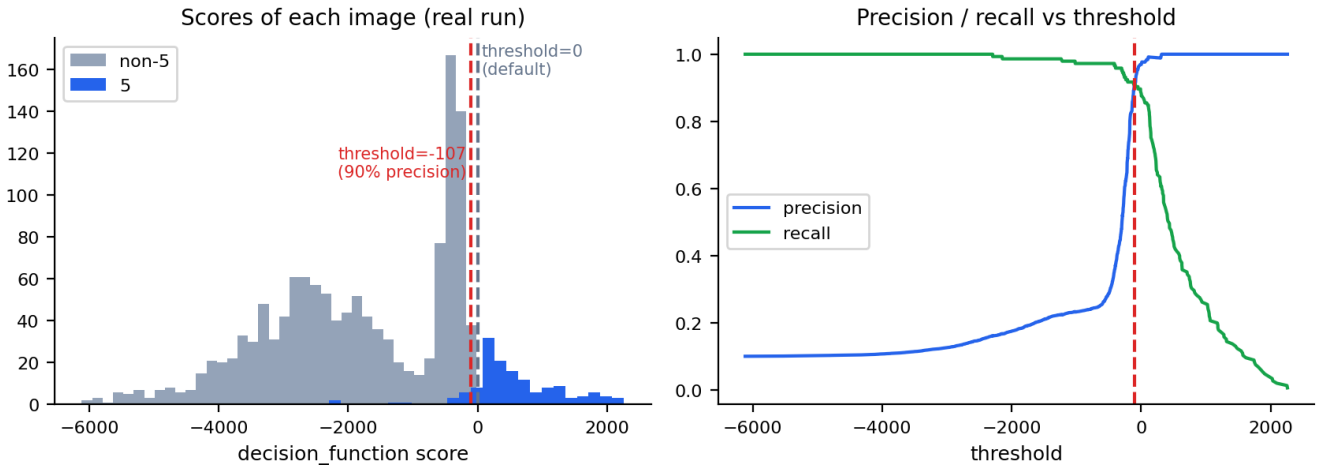
And then we have the Threshold (for example `Threshold = 0`), and the rule:

$$\hat{y} = \text{positive if score} > \text{threshold}, \quad \text{negative otherwise}$$

From your notebook

So if we raise the Threshold to 3000, now the model will be **stricter** and will not say 5 unless the score is very high.

- So when the Threshold increases ← the model will say Positive less often, so False Positives decrease ← Precision increases (usually).
- But on the other hand it may miss real 5 images (False Negatives increase) ← Recall decreases.
- And the opposite is true: if you lower the Threshold ← Recall increases and Precision usually decreases.



From a real run: left: the distribution of the scores, and the dashed lines = different thresholds. Right: precision and recall move opposite to each other when the threshold changes.

```
y_scores = sgd_clf.decision_function([some_digit])
threshold = 0
y_some_digit_pred = (y_scores > threshold)      # True

threshold = 3000          # (in the book; here use a large number)
y_some_digit_pred = (y_scores > threshold)      # False
```

Addition

The same idea with real numbers (`digits`): when we raise the threshold the precision goes up and the recall goes down:

Threshold	Precision	Recall
-1000	23.3%	97.2%
-300	48.1%	95.2%
0	97.0%	88.3%
300	98.9%	61.4%
800	100.0%	29.7%
1500	100.0%	11.7%

From your notebook

`decision_function()`: the book uses it instead of `predict`. The difference: `predict` gives you the final decision `True/False`, whereas `decision_function` gives you the score on which the model built the decision (for example 2164), and then you are the one who applies the threshold:

From your notebook

I myself chose `threshold = 3000` and it became `False` (although the image was really 5) (the Recall went down). The `SGDClassifier` uses `threshold = 0` by default, so `predict()` gives the same result as `(score > 0)`.

From your notebook

Choosing the right Threshold: we do not choose it randomly. First we get the scores for the whole training set using `cross_val_predict` with `method="decision_function"`:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3, method="decision_function")

from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
print(len(precisions), len(recalls), len(thresholds))
```

Output (real run)

```
1437 thresholds → precisions/recalls have one extra element (the last point, precision=1/recall=0)
```

From your notebook

That means instead of `cross_val_predict` returning True/False, we asked it to return the scores. After that `precision_recall_curve` (a function in sklearn) computes Precision and Recall for every possible Threshold; the arrays (`precisions`, `recalls`, `thresholds`) have almost the same length (roughly: `precisions` and `recalls` are one element longer).

Note / Correction

The table you wrote in your notebook (Thresholds: −500, 0, 1000, 2000, 3000 with Precision/Recall) is clearly an **illustrative example** of the idea that Precision increases and Recall decreases. The book's real numbers (MNIST) are: to get precision = 90% you need `threshold ≈ 3370` and the recall becomes about **48%** (not exactly 50%).

```
idx_for_90_precision = (precisions >= 0.90).argmax()      # first index where precision >= 90%
threshold_for_90_precision = thresholds[idx_for_90_precision]
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
print(threshold_for_90_precision)
print(precision_score(y_train_5, y_train_pred_90), recall_score(y_train_5, y_train_pred_90))
```

Output (real run)

```
-106.98
0.9041 0.9103
```

Addition

From the book (MNIST): `threshold = 3370.0195`, `precision = 0.90003`, `recall = 0.47999`. That means with this threshold: when the model says "5" it is right 90% of the time, but it detects only 48% of the real 5s. **And the book warns:** a classifier whose precision is high but whose recall is low is **not useful**. And if someone tells you "let's reach 99% precision" you must ask: at what recall?

On `digits`: `threshold = -107.0` gives 90.4% precision and 91.0% recall (much better than MNIST because `digits` is an easier problem). `argmax()` here returns the first position where it is `True` (the first threshold that achieves precision ≥ 90%).

8 ROC Curve and ROC AUC

From your notebook

ROC Curve: it relates the `TPR` (Recall) to the `FPR` (False Positive Rate). The FPR answers: of all the real Negatives, how many did the model get wrong and call Positive?

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{TNR}$$

From your notebook

`TNR` = **True Negative Rate** means: of all the Negatives, how many did the model identify correctly and call Negative, and it is also called Specificity. And the FPR is related to it: `FPR = 1 - TNR`.

$$\text{TNR (specificity)} = \frac{TN}{TN + FP}$$

From your notebook

- ROC: `X-axis` ← **FPR**, and `Y-axis` ← **TPR (Recall)**.

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

From your notebook

⇒ It computes `FPR + TPR` for different Thresholds. And if the Threshold goes down ⇒ the model will consider more cases Positive, so `TPR` (Recall) and `FPR` both increase (and vice versa).

Note / Correction

In your notebook you wrote "Y-axis" for the ROC twice; the correction: the **X-axis = FPR** and the **Y-axis = TPR**.

From your notebook

ROC AUC: instead of comparing the curves with your eye, you can measure the classifier's performance with one number: the **Area Under the Curve** (the area under the ROC curve), whose values are:

- `1.0` ← perfect classifier.
- `0.5` ← random classifier (the dashed diagonal line).

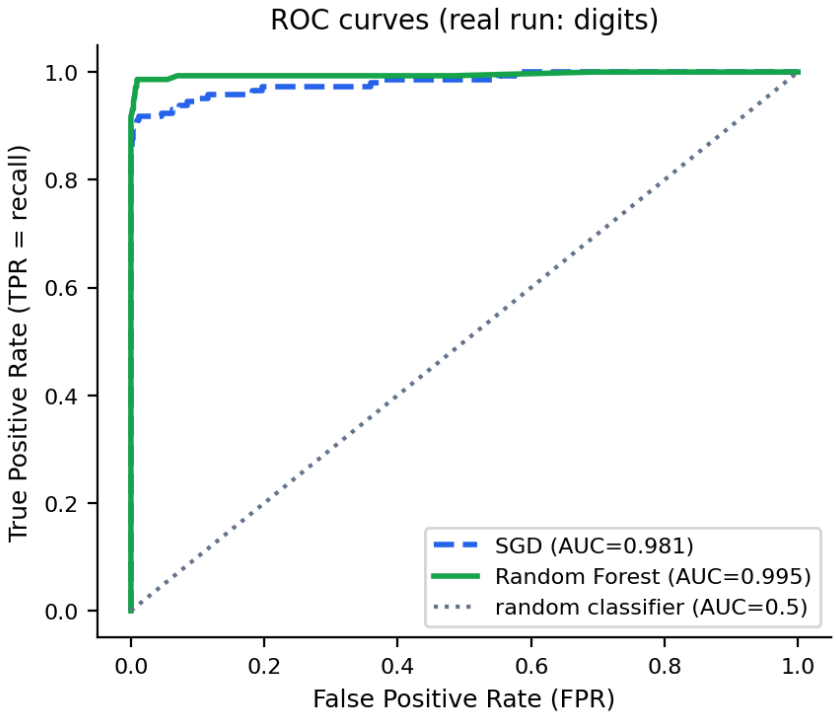
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_train_5, y_scores)
```

Output (real run)

```
0.9807
```

From your notebook

When do I use ROC AUC and when the PR curve? Use the PR curve when the positive class is **rare** or when you care about False Positives more than False Negatives. Otherwise the ROC may be suitable.



The ROC curve (real run): the closer to the top-left corner the better.

Addition

The reason the ROC can mislead when the positives are rare (from the book): in our example the ROC AUC of the SGD = **0.96** "high", but this looks that way because the 5s are few (only 10%) compared with the non-5s, so the FPR stays small even if the model makes many mistakes on the positives. As for the PR curve, it shows you that there is room for improvement. And the closer the curve is to the top-right corner the better.

9 Comparison with Random Forest

From your notebook

Now we will try a different classifier: **Random Forest**. At first we were working with the SGD classifier. The goal here is not to start a new model and be done; the goal is to **compare the performance of the Random Forest with the SGD**.

```
from sklearn.ensemble import RandomForestClassifier
forest_clf = RandomForestClassifier(random_state=42)
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3, method="predict_proba")
print(y_probas_forest[:3])
```

Output (real run)

```
[[0.76 0.24]
 [0.96 0.04]
 [0.03 0.97]]
```

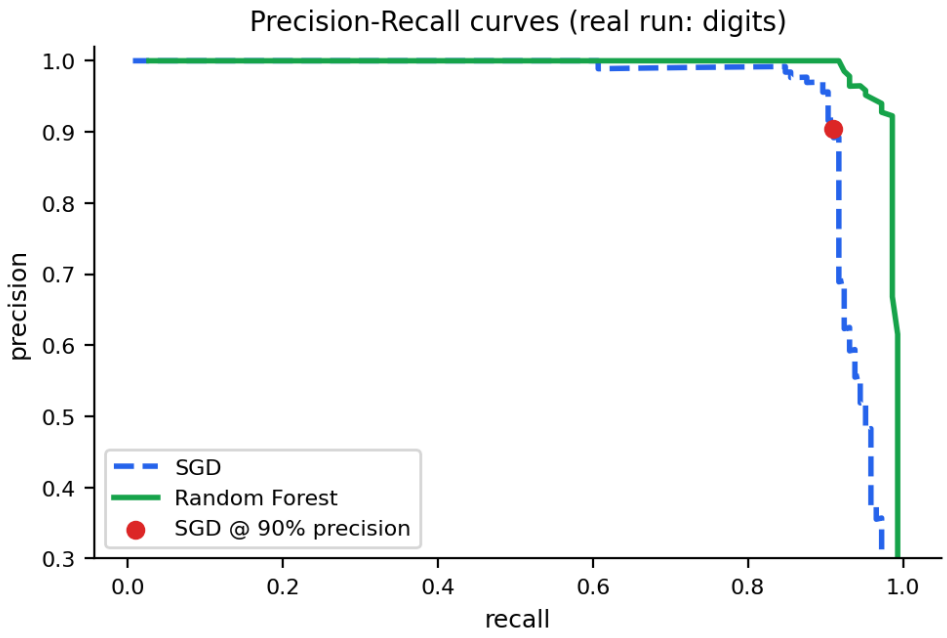
From your notebook

The Random Forest has no `decision_function()`; instead it has `predict_proba()` and returns **probabilities** (a confidence estimate). Instead of returning True/False it returns the probability of each class, for example `[0.11, 0.89]` means: probability of negative = 11% and probability of positive = 89%. So the model sees the probability that it is a 5 = 89%.

From your notebook

`y_probas_forest` looks roughly like: `[[0.11, 0.89], [0.99, 0.01], ...]` ← **Column 0** = probability of negative, **Column 1** = probability of positive. We are interested in 5 = Positive, so we take column number 1 (`[:, 1]`), and this becomes the scores we will use. After that it is the same PR story:

```
y_scores_forest = y_probas_forest[:, 1]         # column 1 from every row = P(positive)
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(y_train_5, y_scores_forest)
```



The Precision-Recall curve (real run): the Random Forest (green) is much closer to the top-right corner than the SGD (dashed blue).

Addition

Real results on `digits`: Random Forest at threshold 50%: F1 = **0.814**, precision = **1.000**, recall = **0.897**, ROC AUC = **0.9952** (versus 0.9807 for the SGD).

Results from the book (MNIST): the PR curve of the Random Forest is much closer to the top-right corner than the SGD. And at threshold 50%: F1 = 0.924 and ROC AUC = 0.998 (versus 0.7325 and 0.960 for the SGD), and roughly precision 99.1% and recall 86.6%.

A warning from the book: these probabilities are **estimated**, not real. For example the images for which the model estimated a probability between 50% and 60% positive were about 94% actually positive, meaning the model was too conservative. And in `sklearn.calibration` there are tools to calibrate the probabilities.

10 Multiclass Classification

From your notebook

We were in Binary classification: 5 or non-5. Now **Multiclass classification** means we have 10 classes (0 → 9); that is, a classifier that can distinguish between more than one class.

From your notebook

Can every classifier do Multiclass? In scikit-learn some classifiers can handle Multiclass directly (natively) like: `LogisticRegression`, `RandomForestClassifier` and `GaussianNB`. And other classifiers are Binary only, like: `SGDClassifier` and `SVC`.

But you can use Binary classifiers to do Multiclass using certain strategies, and this is an important point: scikit-learn automatically converts a Binary classifier into a Multiclass problem.

From your notebook

① **One-vs-Rest (OvR)** (or One-vs-All OvA): this is the first strategy. We have 10 digits. Instead of one classifier that knows all ten, we make **10 Binary classifiers**, each one knows one digit: Classifier 0: 0 vs the rest, Classifier 1: 1 vs the rest, ... and so on. When a new image comes we run all 10 classifiers, each outputs a score, and we choose the classifier that gave the highest score.

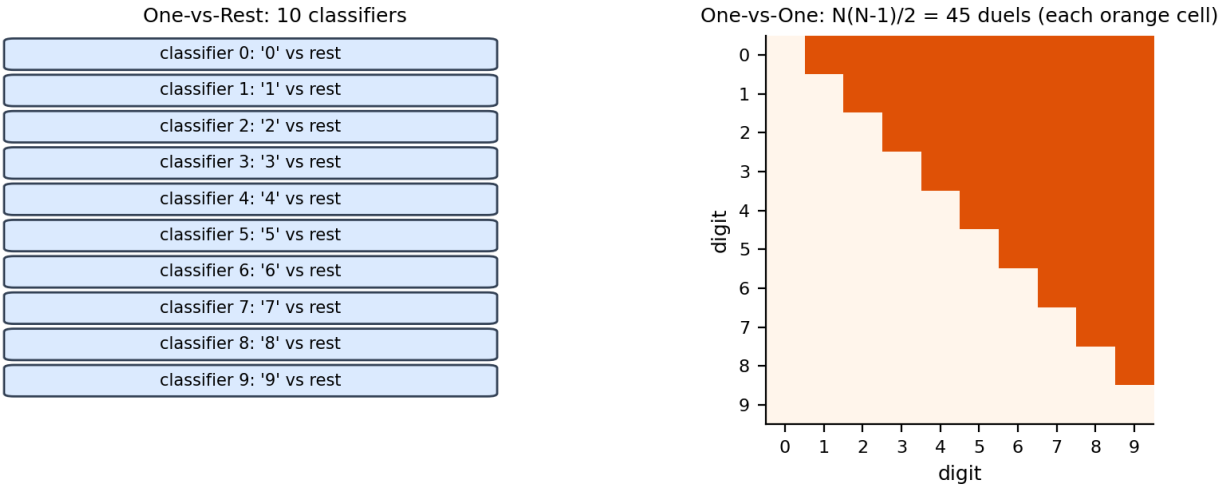
② **One-vs-One (OvO):** the second strategy is different. Instead of one digit against all the others, we make a classifier for **every pair** of digits: 0 vs 1, 0 vs 2, ... 1 vs 2, 1 vs 3, ... 8 vs 9. And if we have N classes the number of classifiers =

$$\text{number of classifiers} = \frac{N(N-1)}{2}$$

From your notebook

In MNIST = **45 Binary classifiers**. And when a new image comes, we pass it to all 45 classifiers, and each classifier does a "duel" between two digits, and the digit that wins the most duels is the final prediction.

So why would I use OvO instead of OvR? In OvO each classifier trains on only a **small part** of the data (only the images of the two digits) so it does not need the whole training set. This is useful with some algorithms (like **SVM**) that become very slow when the training set size grows. So the book says: with some algorithms, training a large number of classifiers on small datasets is faster than training a small number on a large dataset. And with most Binary classifiers, **OvR is the better choice**.



OvR: 10 classifiers (one digit against the rest). OvO: 45 duels (each orange cell = a pair).

From your notebook

If you used a Binary classifier in scikit-learn for a Multiclass problem, it detects that and uses OvO or OvR depending on the algorithm (the book says so).

An SVC example:

```
from sklearn.svm import SVC
svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000])    # y_train NOT y_train_5: the original 10-class target
print(svm_clf.predict([some_digit]))
some_digit_scores = svm_clf.decision_function([some_digit])
print(some_digit_scores.round(2))
class_id = some_digit_scores.argmax()          # index of the biggest score
print(class_id, svm_clf.classes_[class_id])    # classes_[class_id] = the actual class label
```

Output (real run)

```
[5]      # (on digits we trained on the first 600 images)
```

From your notebook

Notice that it said `y_train` and not `y_train_5`, because we are doing Multiclass for the original problem (0 → 9).

And since the SVC is a Binary classifier (5 or not 5), scikit-learn used OvO and trained 45 Binary classifiers. The result: `array(['5'])` means the image was classified as 5, but internally it was not one classifier that said 5; there were 45 duels and the 5 won the largest number of them.

From your notebook

`decision_function()` **in Multiclass:** with the Binary classifier you got one score: is it 5? But here we have 10 classes so it returns 10 scores (a score for each class):

Output (real run)

```
[[ 6.14  1.89  2.85  7.23  0.72  9.28 -0.29  3.99  5.18  8.27]]
5 5
```

From your notebook

The book showed: `[..., 3.79, ..., 0.73, ...]` ← 10 scores, and the highest score was 9.30 and it is the index number 5, so the Prediction = 5. `argmax()` returns the index of the largest value. And then `classes_[class_id]` gets the actual label (in MNIST the index matches the digit by coincidence, but in general you must do a lookup).

Addition

And if you want to see the 45 duels themselves ask for `SVC(decision_function_shape="ovo")`: the shape becomes (1, 45) (45 scores) instead of (1, 10) (10 scores). The score in OvO (from the book): each class gets score = the number of duels it won plus/minus a small adjustment (≤ 0.33) to break ties.

From your notebook

What if I want to force OvO or OvR myself (instead of scikit-learn deciding)?

```
from sklearn.multiclass import OneVsRestClassifier       # or OneVsOneClassifier
ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
print(len(ovr_clf.estimators_))
```

Output (real run)

```
10
```

From your notebook

Here I said explicitly: use SVC but convert it to Multiclass using OvR (so it trains 10 classifiers).

Addition

The `SGDClassifier` automatically uses OvR with Multiclass: `sgd_clf.fit(X_train, y_train)` gives 10 scores.

From the book (MNIST): this model predicts `'3'` for `some_digit` wrongly (although it is 5), and with `cross_val_score` it gets an accuracy of about 87% (`0.87365, 0.85835, 0.8689`); and if you do `StandardScaler` first it rises to about 90% (`0.8983, 0.891, 0.9018`). So scaling matters here too.

On `digits` : accuracy ≈ 92.2% without scaling and ≈ 94.3% after `StandardScaler` . And for comparison: random guessing with 10 classes = 10% only.

11 Error Analysis

```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)

from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
# better: add normalize and values_format
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, normalize="true", values_format=".0%")
```

From your notebook

Error Analysis: it is when we ask the model: what exactly does it get wrong? That is, analyzing the types of errors instead of looking only at the accuracy number.

In Multiclass, the CM is no longer TN/FP/FN/TP like the Binary one, because we now have 10 classes, so the CM will be 10 × 10: `Row = Actual class` , `Columns = Predicted class` .

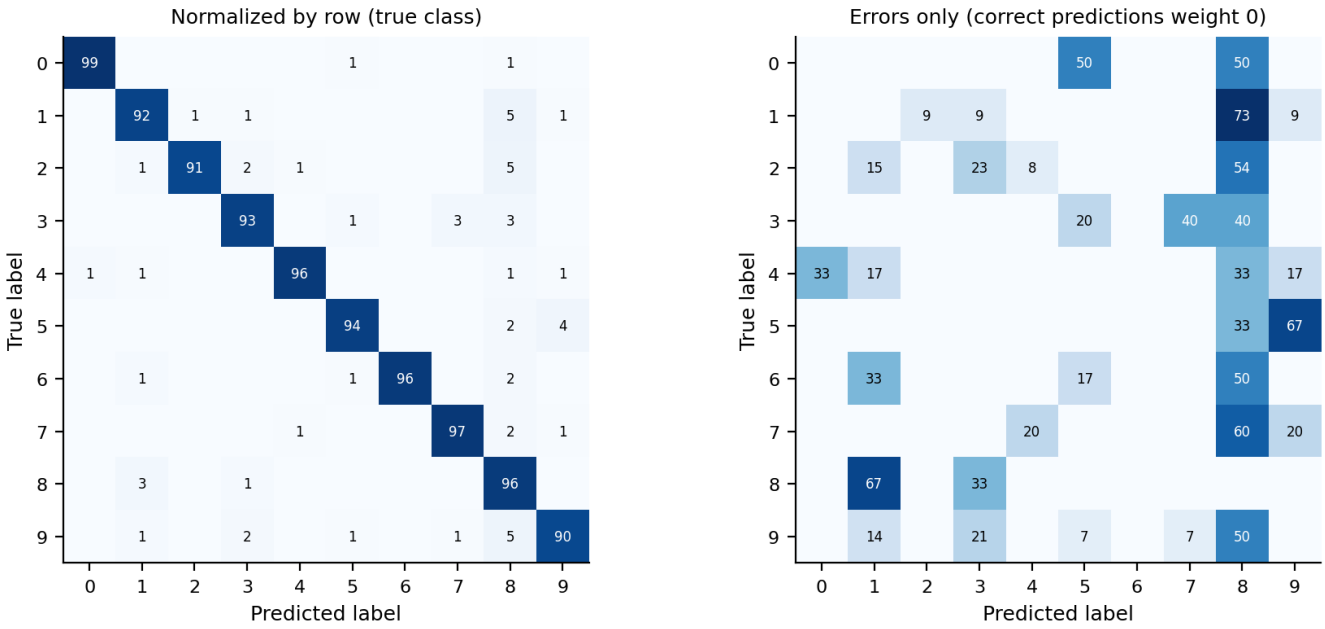
The book does it with `ConfusionMatrixDisplay` :

From your notebook

It draws the CM as a picture instead of dealing with many numbers.

How do you read the CM? The numbers on the **Main Diagonal** = the correct predictions. And the darker the Main Diagonal, the better the model in general.

`normalize="true"` + `values_format=".0%"` : converts the numbers into percentages (divided by the row total) so they are clearer, because some classes have more samples than others.



Real run on `digits` (overall accuracy 94%): left the percentages per class, right the errors only (the diagonal = zero). The biggest error: the digit 1 gets confused with 8 (5%).

Addition

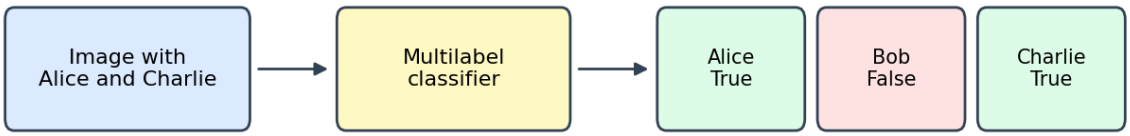
What the book discovered from the CM after normalizing on MNIST:

- Only **82%** of the 5 images were classified correctly. And the biggest error with the 5 was classifying it as 8 (it happened with **10%** of the 5s). But only 2% of the 8s were classified as 5; that means the confusion matrix is **not symmetrical**.
- If you want the errors to show more: `sample_weight=(y_train_pred != y_train)` puts a zero weight on the correct predictions. Here the column of the **8** becomes very bright, meaning many digits were classified as 8 wrongly (the most common error for most classes).
- **Note:** the percentages in the "errors only" version are not a percentage of all images; for example 36% in the row 7 column 9 means 36% of the 7's **errors** were classified as 9, not 36% of all the 7s. (The truth is only 3% of the 7s were classified 9.) And `normalize="pred"` normalizes on the column.
- The conclusion in the book: focus on reducing the false 8s by gathering more data of digits that look like an 8 (but are not 8) so the model learns to tell them apart, or build new features (like the number of closed loops), or do preprocessing on the images.

12 Multilabel Classification

From your notebook

What if I have an image and want to recognize more than one thing? For example: I have a classifier that recognizes 3 people (Alice, Bob, Charlie), and an image containing Alice and Charlie, the result `[True, False, True]`. So a single instance can come out with more than one Binary label at the same time = **Multilabel classification**.



One image → more than one label (each label has a value True/False).

From your notebook

The MNIST example from the book: it made a label for each image: the first label: is the digit **large**? And the second label: is the digit **odd**?

```
import numpy as np
y_train_large = (y_train >= '7')                    # 7, 8, 9 -> True | 0..6 -> False
y_train_odd = (y_train.astype('int8') % 2 == 1)     # 1,3,5,7,9 -> True | 0,2,4,6,8 -> False
y_multilabel = np.c_[y_train_large, y_train_odd]     # stack the 2 labels as 2 columns
```

From your notebook

For example for the digit 5: `large = False` and `odd = True` ← `[False, True]`. So now I have 2 targets for each image instead of one target.

```
from sklearn.neighbors import KNeighborsClassifier
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
print(knn_clf.predict([some_digit]))
```

Output (real run)

```
[[False  True]]      # 5 is NOT large, and it IS odd ✓
```

From your notebook

Here the KNN learns from `X_train` → `[Large?, Odd?]` and not from a single number.

Evaluating a Multilabel classifier: there is no single metric you must always use; choosing the metric depends on the project. The book suggested: compute F1 for each label and then take the average:

```
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
f1_score(y_multilabel, y_train_knn_pred, average="macro")
```

Output (real run)

```
0.9780      # (book on MNIST: 0.9764)
```

From your notebook

- `average="macro"` : compute F1 for each label then take the mean, and it gives every label **the same weight**.
- The problem with macro: if the number of cases differs a lot (for example many Alice images, few Bob images, and very few Charlie images) you may want to give the label with more cases a bigger weight. So you use `average="weighted"` : each label takes its weight according to its **support** (i.e. the number of instances that belong to it).

Addition

On `digits` : F1 for each label = 0.967 (large) and 0.989 (odd). The macro = **0.9780** and the weighted = **0.9808**; close because the labels are almost balanced. The difference grows when a label is rare.

From your notebook

What if the classifier does not support Multilabel directly? (SVC for example). You can train a separate model for each label, but then a problem appears: the models know nothing about each other (they do not see what the other models predicted). To solve the problem, the book suggests organizing the models in a **Chain**: **Classifier Chain**.

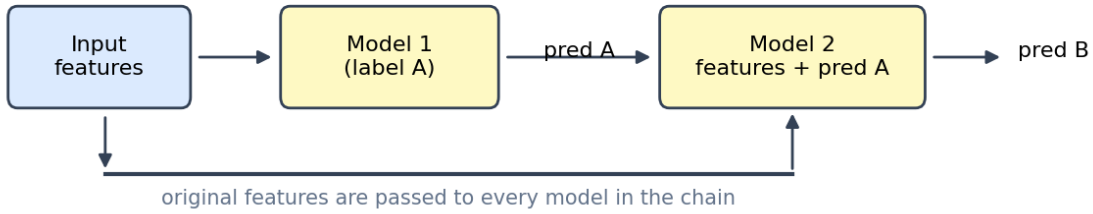
The idea: Input ← Model 1 ← Prediction 1; and the second model depends on: original features + predictions from previous models. So the models in the chain can benefit from the relationships between the labels.

```
from sklearn.multioutput import ClassifierChain
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
print(chain_clf.predict([some_digit]))
```

```
Output (real run)
[[False  True]]
```

From your notebook

Here we created a Classifier Chain using SVC, and `cv=3` makes the chain use cross-validation to obtain clean out-of-sample predictions during training, instead of depending directly on results coming from the same data the previous models trained on.



Classifier Chain: each model takes the original features + the predictions of the models before it.

13 Multioutput Classification

From your notebook

Multioutput–multiclass classification is a generalization of Multilabel classification. The basic difference: in Multilabel each label has **only two values** (True/False), but in Multioutput each output can have **more than two values** (that is, each label itself is Multiclass).

From your notebook

A book example: Image Denoising: the input = a noisy image, and the output = a clean image. The model is trained to do: Noisy Image → Clean Image. The output has 784 pixels, and each pixel is a separate output, and each output can take a value from 0 to 255 (that is, many values). So we have Multiple outputs and each output has many values: **Multioutput classification**.

```
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise                  # noisy images = the INPUT
y_train_mod = X_train                          # clean images = the OUTPUT (784 outputs!)

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
```



Real run on `digits` (KNN, errors by MSE): after cleaning the error dropped from 9.1 to 3.4. (Row 1: noisy, row 2: the model output, row 3: the required clean.)

Type	Number of outputs	Values of each output	Example
Binary	1	two values	5 or not
Multiclass	1	more than two values	a digit from 0 to 9
Multilabel	more than 1	two values for each	Alice, Bob, Charlie (True/False)
Multioutput	more than 1	more than two values for each	each pixel (0-255) in the clean image

Addition

The line between Classification and Regression is blurry here: predicting a pixel value is closer to regression, but the book considers it classification because it is multioutput and the values are limited.

14 Quick review questions

Answer first in your head, then check the answers on the next page.

1. What is the difference between Classification and Regression?
2. Why did we convert the labels to (True/False) `y_train_5`?
3. What is the SGDClassifier and why is it suitable for big data?
4. Why is accuracy not a good metric for classifiers? An example (dummy).
5. What is the difference between cross_val_score and cross_val_predict?
6. Explain the Confusion Matrix (TN, FP, FN, TP).
7. Write the equation of Precision and Recall and explain them.
8. An example of how you can achieve 100% precision with a bad model?
9. Why does F1 use the harmonic mean?
10. When do I prefer Precision, and when Recall?
11. Explain the Precision/Recall trade-off and the threshold.
12. The difference between predict and decision_function?
13. How do you choose a threshold for 90% precision?
14. What is the ROC curve? And what do FPR and TNR mean?
15. What do ROC AUC = 1 and 0.5 mean?
16. When do I use the PR curve instead of ROC?
17. What is the difference between decision_function and predict_proba? And how do we compare Random Forest with the SGD?
18. List 3 classifiers that support multiclass directly and two that are binary only.
19. Explain OvR and OvO and how many classifiers in MNIST?
20. How do you get the class from decision_function in multiclass?
21. What is Error Analysis? And how do you read the CM?
22. What is the difference between Multilabel and Multioutput?
23. How do you evaluate Multilabel? And when macro and when weighted?
24. What is a Classifier Chain?

15 Review question answers

1. Classification predicts a class, and Regression predicts a number.
2. To build a binary classifier (5 or not 5), so the model only needs to know whether the image is a 5.
3. It learns from the instances one by one (Stochastic Gradient Descent), so it is suitable for large datasets and online learning.
4. With imbalanced (skewed) data: a dummy classifier that always says "not 5" gets an accuracy of about 90% (0.90965 in the book) without learning anything.
5. The first returns scores per fold, and the second returns out-of-sample predictions for every instance.
6. The rows = reality and the columns = prediction. TN and TP are correct, FP is a false alarm, FN is a missed positive case.
7. precision = TP/(TP+FP): of those it called 5, how many really were 5. recall = TP/(TP+FN): of the real 5s, how many it detected.
8. A model that predicts positive for only one image it is very sure about and is right: TP=1, FP=0, but it misses the rest of the 5s (very low recall).
9. To penalize it if precision or recall is very low; both must be high.
10. Precision if a false positive is costly (safe videos for children). Recall if a false negative is costly (theft/disease detection).
11. The model computes a score and compares it with a threshold. Raising it = precision usually increases and recall decreases, and lowering it is the opposite.
12. predict returns the final decision (True/False), and decision_function returns the score and you set the threshold.
13. cross_val_predict(method="decision_function") then precision_recall_curve, then argmax() on precisions >= 0.90 (book: threshold ≈ 3370 → recall ≈ 48%).
14. A curve of TPR (Y) against FPR (X). FPR = FP/(FP+TN) = 1 − TNR, and TNR = specificity.
15. 1 = perfect classifier, 0.5 = random.
16. When the positive class is rare or when false positives matter more than false negatives.
17. Random Forest has no decision_function, so we use predict_proba[:, 1] as scores and compare PR/ROC.
18. Natively: LogisticRegression, RandomForestClassifier, GaussianNB. Binary only: SGDClassifier, SVC.
19. OvR: 10 classifiers (one digit against the rest). OvO: N(N−1)/2 = 45 (pair against pair). OvO is better with SVM because it trains on small parts.
20. argmax() gives the index, and classes_[class_id] gives the label.
21. Analyzing the types of errors. The Main Diagonal = the correct predictions; we do normalize="true" and remove the diagonal (sample_weight) to see the errors.
22. Multilabel: each label has two values (True/False). Multioutput: each output can be multiclass (like each pixel 0-255).

23. F1 for each label then the mean. macro = same weight, weighted = according to the number of cases (support) if the labels are imbalanced.
24. A chain of models: each model takes the original features + the predictions of the one before it, so it benefits from the relationships between the labels.